

## **Instructions**

This is a case study. It is based on a real product and a real set of challenges.

There are no trick questions and no single correct answer. We are not looking for the 'right' solution — we are looking for how you think: how you frame problems, how you reason through trade-offs, how you communicate with stakeholders, and how you approach ambiguity.

*FYI:* All ideation and deliverables for this challenge remain your intellectual property. Bitso will not use your challenge resolution in any ongoing or future projects.

## **The Situation**

A digital financial services company operates across 5 countries with over 150 currencies. At the center of its operations is a concept called solvency — at any given moment, the company must hold at least as much money as it owes to its users. This is both a regulatory requirement and a fundamental matter of customer trust.

To verify solvency continuously, the company built an internal system called the AUM Reconciliation platform (AUM = Assets Under Management). The system runs on a schedule, compares everything the company has (funds held in 19+ payment integrations: crypto custodians, international payment rails, bank accounts, blockchain wallets) against everything it owes (user balances, pending withdrawals, accrued fees, savings and margin obligations), and produces a discrepancy report per currency. When the discrepancy exceeds acceptable thresholds, alerts are sent to the relevant teams. The system is production-grade, actively used, and critical to daily operations.

## **The Problem**

### ***The architecture has hit its limits***

The current version of the platform (internally called V2) was built with a direct integration model: the reconciliation system itself reaches out to all 19+ providers at reconciliation time to fetch current balances. This creates a set of compounding problems:

- Accuracy gaps: Different providers respond at different speeds. Since balances and liabilities must be compared at the same point in time, timing misalignment

between provider responses introduces systemic inaccuracy. The system applies workaround strategies (timestamp heuristics) to compensate, but these are approximations.

- Timeliness constraints: Reports reflect the state of funds with some lag. The lag is not large, but it is real and felt by the teams that depend on these reports daily.
- Operational overhead: Onboarding a new payment provider requires engineering changes to the reconciliation service itself — new integration code, business rule configuration, testing, and deployment.

### **A new architecture has been decided but not delivered**

Engineering and product leadership have agreed on a next-generation architecture (internally called V3). The direction:

Reconcilable Services — domain-specific microservices, one per payment integration — take ownership of polling their respective providers and publishing standardized balance snapshot events to a shared message stream.

The Reconciliation Engine consumes these events and uses them to generate solvency reports, decoupled from direct provider integrations. This architectural shift is expected to improve accuracy (snapshots are taken at the source with precise timestamps), timeliness (a continuous event stream replaces scheduled polling), and onboarding efficiency (new providers publish their own events; the engine doesn't change). V3 has no delivery estimate.

The previous PM was unable to scope it before leaving. The system spans 150+ currencies across 5 countries, is a live production system, and the migration touches multiple engineering teams.

### **The stakeholder pressure**

The payment operations team — the primary consumers of solvency reports — is frustrated. They want a near-real-time, near-100% accurate view of the company's solvency position. They escalate frequently when reports feel stale or when discrepancies appear that they can't explain. Their trust in the current system is fragile.

### **Your Team**

You would work with a dedicated engineering team:

- 1 Engineering Manager
- 5 Senior Engineers
- 1 Staff Engineer

The team is experienced and has deep domain knowledge. V3 is technically well-understood by engineering — the challenge is product definition, scoping, migration sequencing, and stakeholder alignment.

## **Questions**

### **Question 1 — Stakeholder Management**

The payment operations team requires a solvency system that is near-real-time and nearly 100% accurate. Engineering has indicated that this is unachievable in any distributed system at this scale—snapshots are slightly historical when computed, and perfect accuracy conflicts with timeliness. How do you address this?

*Consider:* How do you engage the operations team? What do you commit to and what do you not? How do you rebuild trust without overpromising what V2 or V3 can deliver? What short-term actions reduce their anxiety before V3 is ready?

### **Question 2 — Scoping Under Ambiguity**

V3 lacks an estimate. Engineering understands the architecture, but no one has scoped delivery. The system covers 150+ currencies, 5 countries, 19+ provider integrations, and must remain online during migration. How do you approach scoping this initiative?

*Consider:* What process do you follow? Who do you involve? What questions must be answered to produce a credible roadmap? What is the scoping output? How do you communicate confidence levels to leadership amid high uncertainty?

### **Question 3 — Prioritization**

You now manage both V2 (in production, used daily by payment operations) and V3 (a major architectural initiative with undefined scope and high strategic importance). How do you prioritize between them?

*Consider:* What would pause V3 to focus on V2? What would accelerate V3 at V2's expense? Can work be structured so that both progress in parallel? How do you communicate prioritization logic to stakeholders with competing interests?

#### **Question 4 – Defining Success**

Assuming V3 is delivered, how do you determine it was worth building? What defines success for this product, and how would you measure it?

*Consider:* What metrics would you define? What would "better" mean compared to V2? How do you measure accuracy, timeliness, and operational efficiency meaningfully for engineering and business? Could V3 worsen any aspects while improving others?